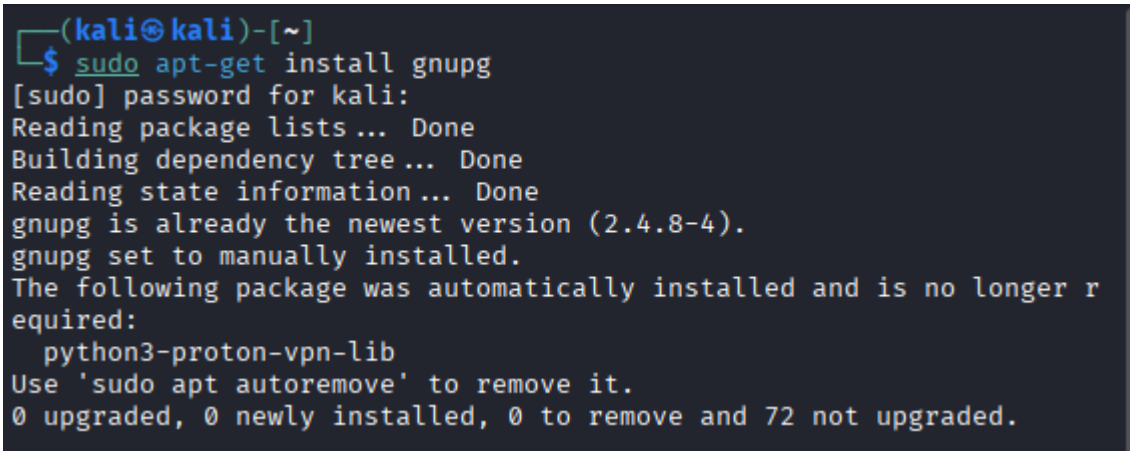
# Uso de GPG en el terminal

### 1. Instalación de GPG

Para instalar GPG en sistemas Linux (basados en Debian), simplemente ejecuta:

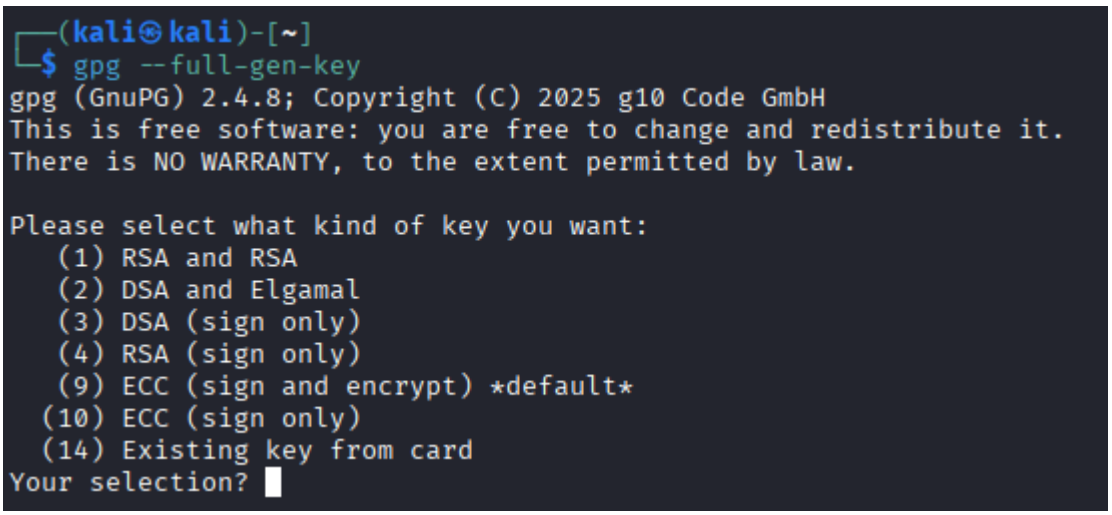
```
sudo apt-get install gnupg
```




### 2. Creación de una clave GPG

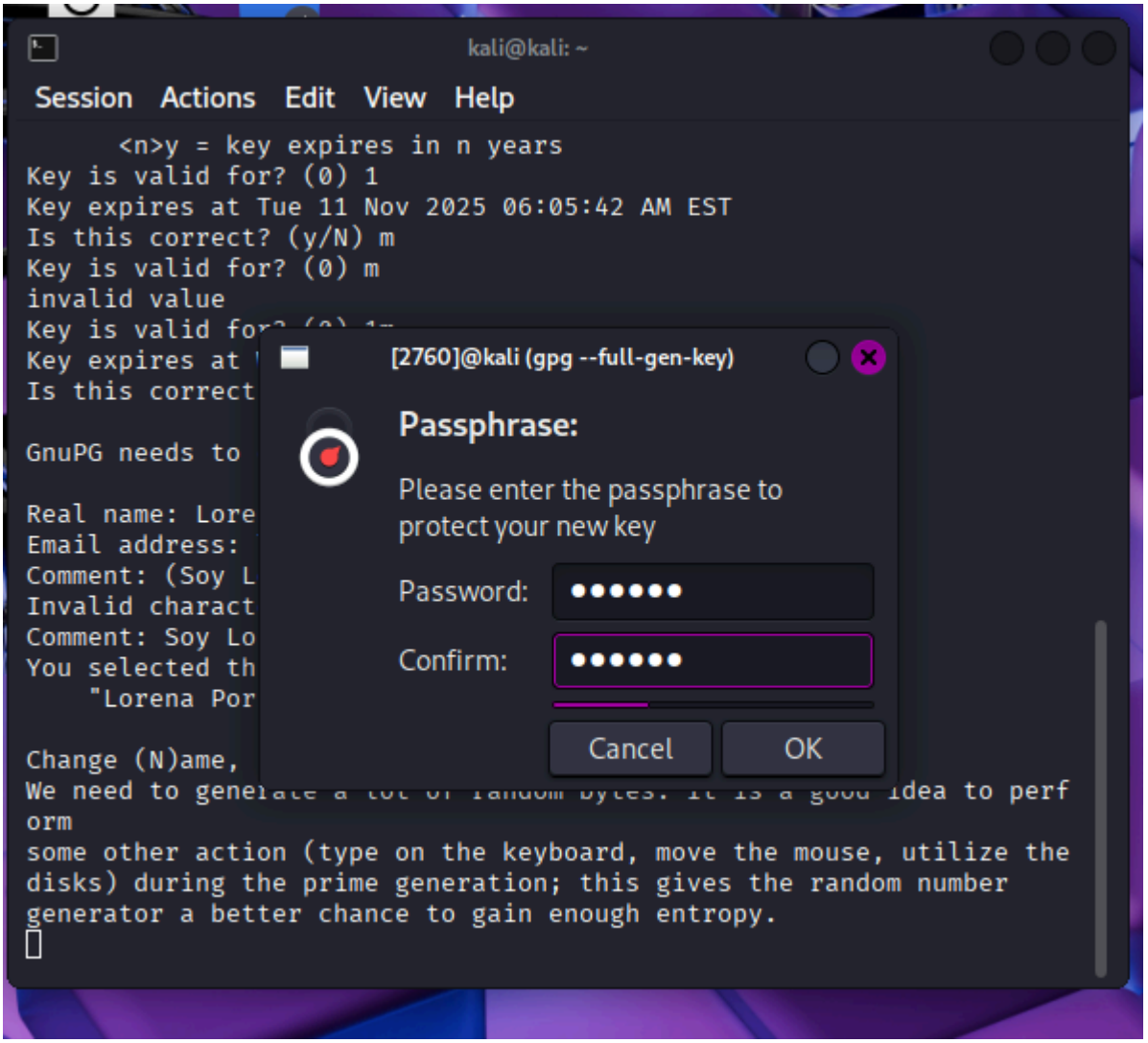
Supón el nombre Carlos Sánchez y el correo electrónico carlos@ejemplo.com.
El comando para generar la clave es:

```
gpg --full-gen-key
```




Este te guiará a través de varios pasos. Usar los siguientes parámetros:

- **Tipo de clave:** RSA y RSA *(opción por defecto, selecciona ‘1’)*
- **Tamaño de la clave:** 4096 bits *(para mayor seguridad)*
- **Duración de la clave:** 1y *(puedes definir la duración o elegir que nunca expire)*
- **Nombre completo:** Carlos Sánchez
- **Correo electrónico:** carlos@ejemplo.com
- **Comentario:** (Soy Carlos)
- **Contraseña:** Carlos1234
-




**Password= lorena**

```
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 1
Key expires at Tue 11 Nov 2025 06:05:42 AM EST
Is this correct? (y/N) m
Key is valid for? (0) m
invalid value
Key is valid for? (0) 1m
Key expires at Wed 10 Dec 2025 06:06:29 AM EST
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Lorena Porras
Email address: lorenap@ejemplo.com
Comment: (Soy Lorena)
Invalid character in comment
Comment: Soy Lorena
You selected this USER-ID:
    "Lorena Porras (Soy Lorena) <lorenap@ejemplo.com>"
```

Si prefieres usar el modo experto para más control, ejecuta:

```
gpg --full-gen-key --expert
```

Aquí podrás seleccionar algoritmos como ECC *(criptografía de curva elíptica)*, menos común pero más eficiente.

```
┌──(kali㉿kali)-[~]
└─$ gpg --full-gen-key --expert
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (7) DSA (set your own capabilities)
   (8) RSA (set your own capabilities)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sign only)
  (11) ECC (set your own capabilities)
  (13) Existing key
  (14) Existing key from card
Your selection? 9
Please select which elliptic curve you want:
   (1) Curve 25519 *default*
   (2) Curve 448
   (3) NIST P-256
   (4) NIST P-384
   (5) NIST P-521
```

```
  (13) Existing key
  (14) Existing key from card
Your selection? 9
Please select which elliptic curve you want:
   (1) Curve 25519 *default*
   (2) Curve 448
   (3) NIST P-256
   (4) NIST P-384
   (5) NIST P-521
   (6) Brainpool P-256
   (7) Brainpool P-384
   (8) Brainpool P-512
   (9) secp256k1
Your selection? 6
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 1m
Key expires at Wed 10 Dec 2025 06:10:32 AM EST
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.
```

```
Real name: Lore P
Email address: lorep@example.com
Comment: lore
You selected this USER-ID:
    "Lore P (lore) <lorep@example.com>"

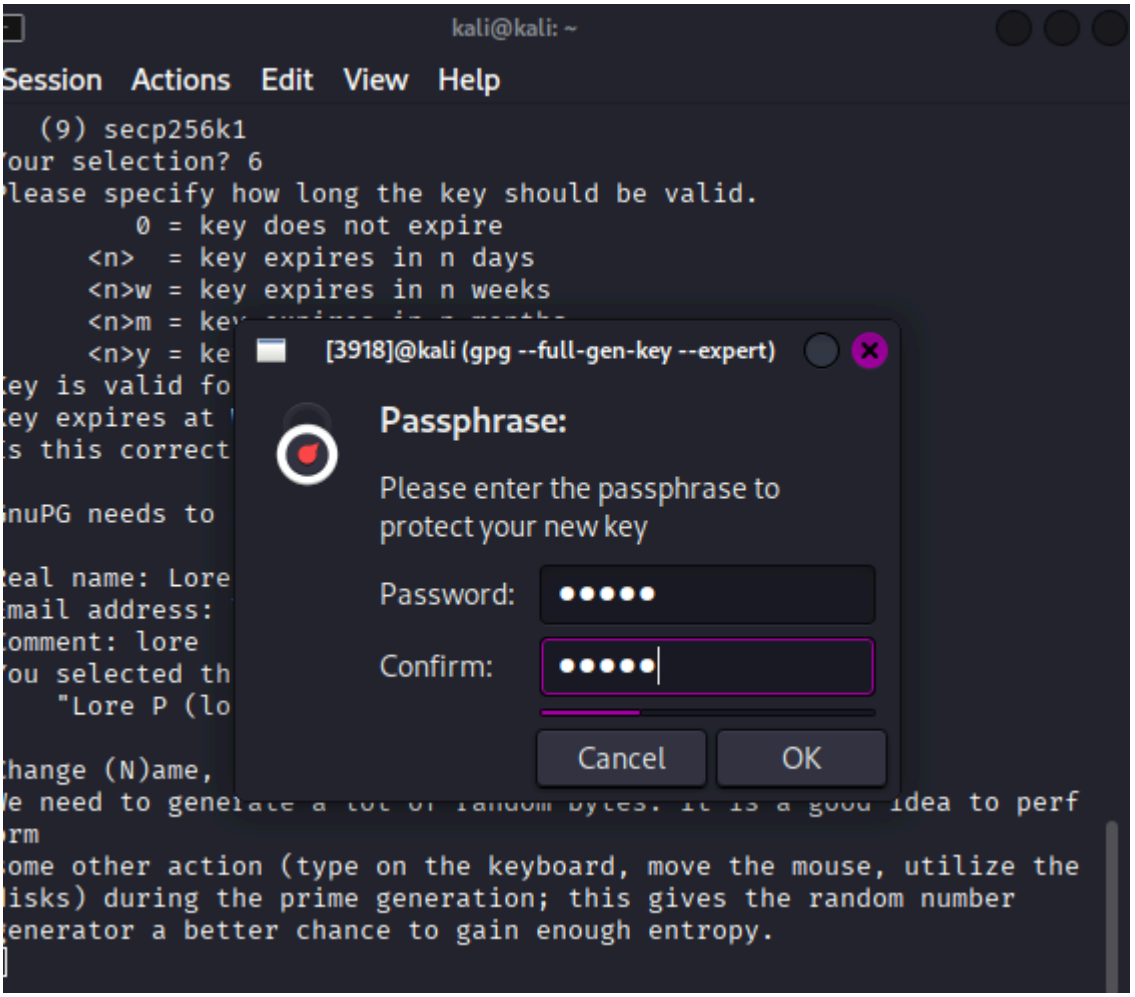
Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perf
orm
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perf
orm
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/home/kali/.gnupg/openpgp-rev
ocs.d/CFFF0798D1D543D5894B73651923F52FF55E6DF9.rev'
public and secret key created and signed.

pub   brainpoolP256r1 2025-11-10 [SC] [expires: 2025-12-10]
      CFFF0798D1D543D5894B73651923F52FF55E6DF9
uid                      Lore P (lore) <lorep@example.com>
sub   brainpoolP256r1 2025-11-10 [E] [expires: 2025-12-10]
```

```
Comment: lore
You selected this USER-ID:
    "Lore P (lore) <lorep@example.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perf
orm
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perf
orm
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/home/kali/.gnupg/openpgp-rev
ocs.d/CFFF0798D1D543D5894B73651923F52FF55E6DF9.rev'
public and secret key created and signed.

pub   brainpoolP256r1 2025-11-10 [SC] [expires: 2025-12-10]
      CFFF0798D1D543D5894B73651923F52FF55E6DF9
uid                      Lore P (lore) <lorep@example.com>
sub   brainpoolP256r1 2025-11-10 [E] [expires: 2025-12-10]
```



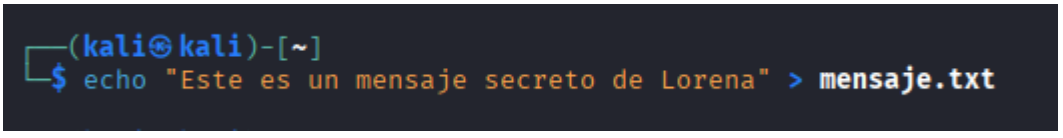

**Password: lorep**

---

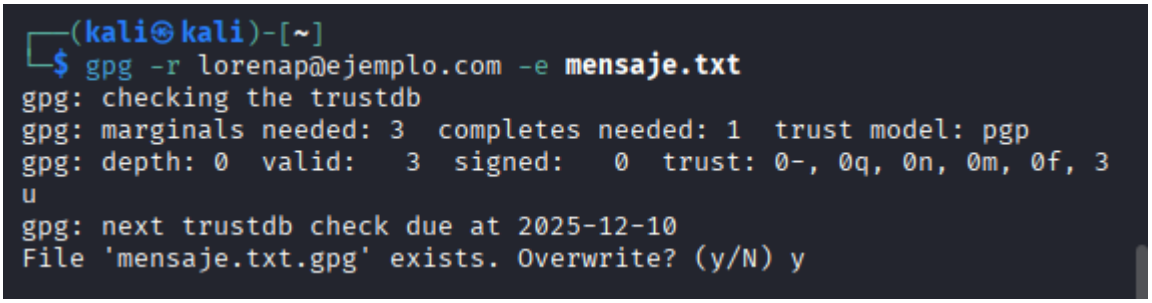
### 3. Cifrado de un archivo de texto

Creamos el archivo de texto llamado mensaje.txt con el siguiente contenido:

```
echo "Este es un mensaje secreto de Carlos" > mensaje.txt
```




Cifraremos el archivo con:




```
gpg -r carlos@ejemplo.com -e mensaje.txt
```

Esto creará un archivo cifrado llamado mensaje.txt.gpg.
El parámetro -r especifica el destinatario (clave pública de Juan), y -e indica que se debe cifrar el archivo.

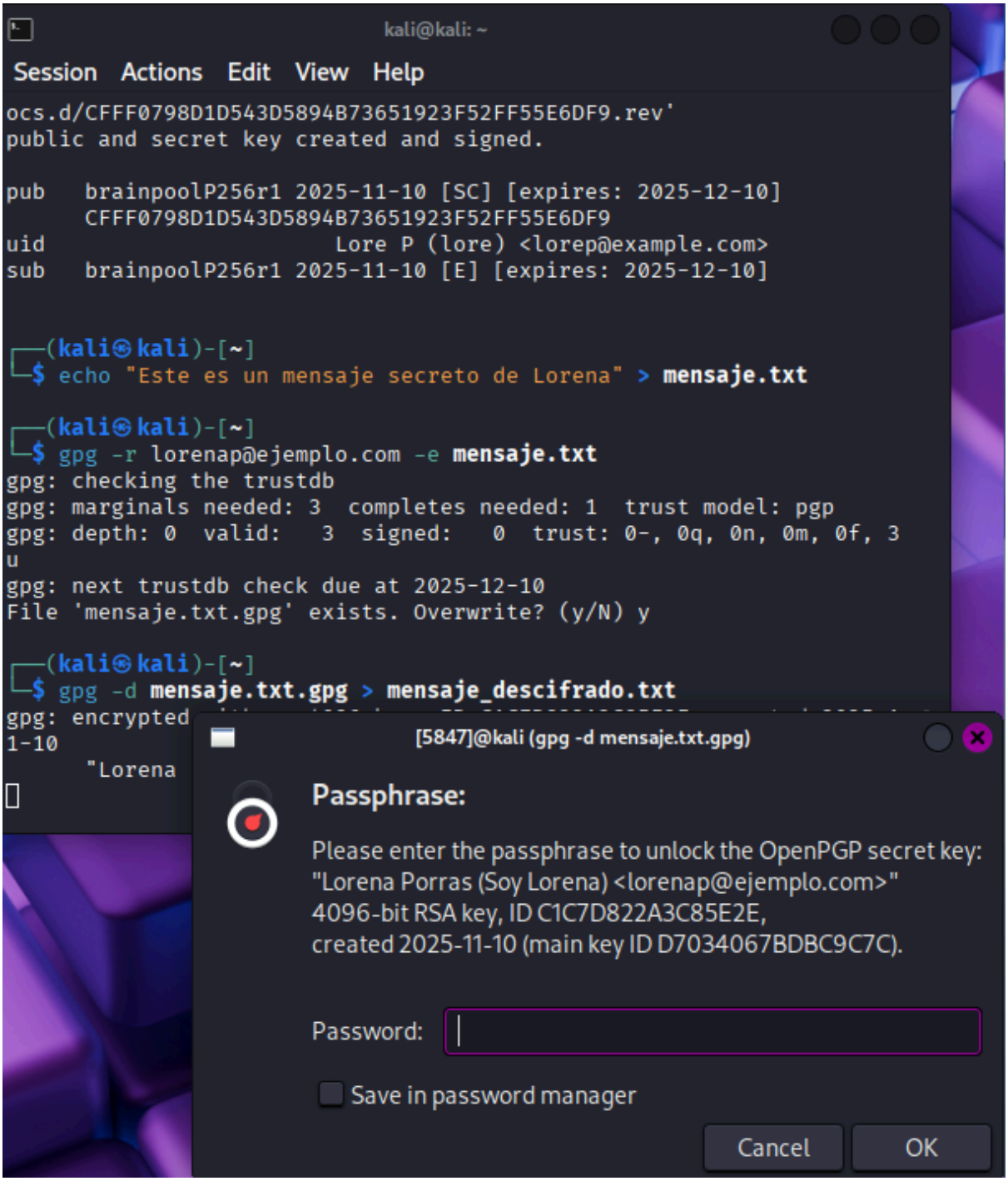
---

### 4. Descifrado de un archivo

El archivo mensaje.txt.gpg podrá descifrarse con su clave privada usando el siguiente comando:

```
gpg -d mensaje.txt.gpg > mensaje_descifrado.txt
```

El archivo mensaje_descifrado.txt contendrá el texto original que Carlos envió.




```
(kali㉿kali)-[~]
$ gpg -d mensaje.txt.gpg > mensaje_descifrado.txt
gpg: encrypted with rsa4096 key, ID C1C7D822A3C85E2E, created 2025-1
1-10
      "Lorena Porras (Soy Lorena) <lorenap@ejemplo.com>"

(kali㉿kali)-[~]
$
```

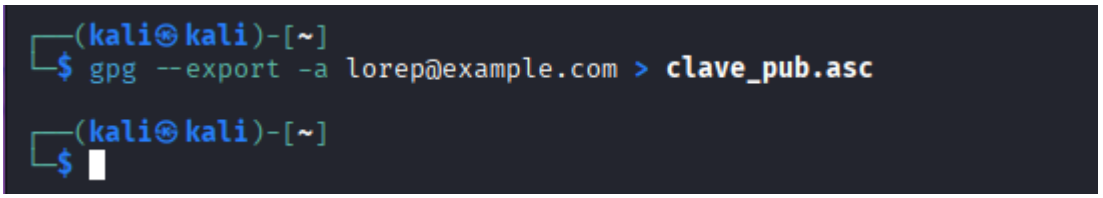
---

### 5. Exportación de claves

Carlos quiere compartir su clave pública para que otros puedan cifrar archivos que solo él pueda descifrar. Para exportar su clave pública, usa:

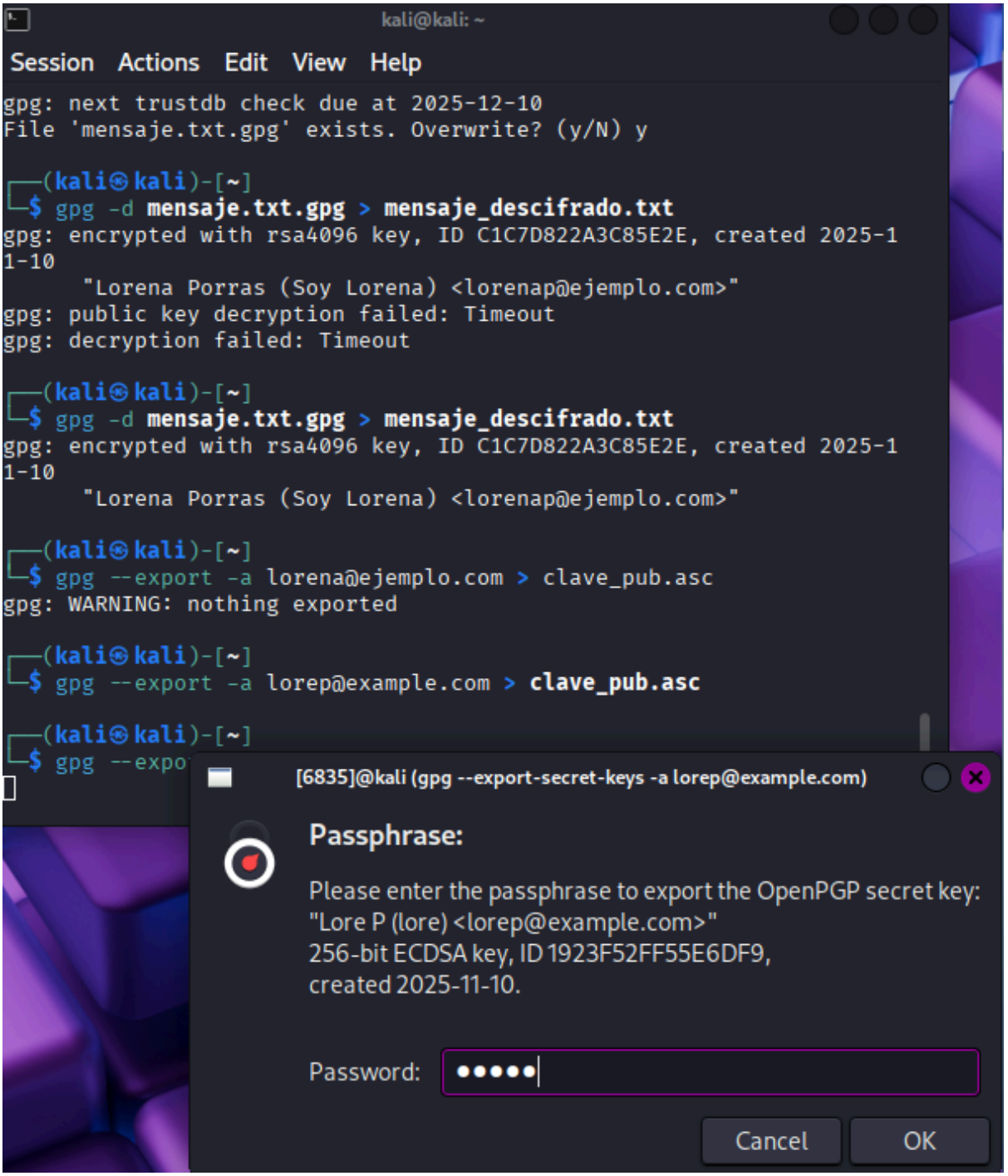
```
gpg --export -a carlos@ejemplo.com > clave_pub.asc

Esto creará el archivo clave_publica_carlos.asc, que contiene su clave pública en formato ASCII.
```




Si Carlos quiere hacer una copia de seguridad de su clave privada, ejecuta:

```
gpg --export-secret-keys -a carlos@ejemplo.com > clave_priv.asc
```




```
(kali㉿kali)-[~]
$ gpg --export -a lorep@example.com > clave_pub.asc

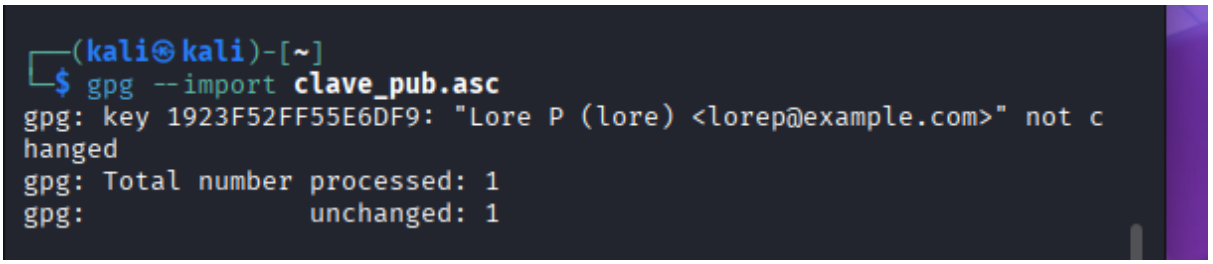
(kali㉿kali)-[~]
$ gpg --export-secret-keys -a lorep@example.com > clave_priv.asc
```

### 6. Importación de claves

Cuando alguien reciba la clave pública de Carlos, podrá importarla en su sistema con el siguiente comando:

```
gpg --import clave_pub.asc
```

Esto añadirá la clave pública de Carlos a su llavero de GPG.




---

### 7. Opciones de cifrado en modo experto

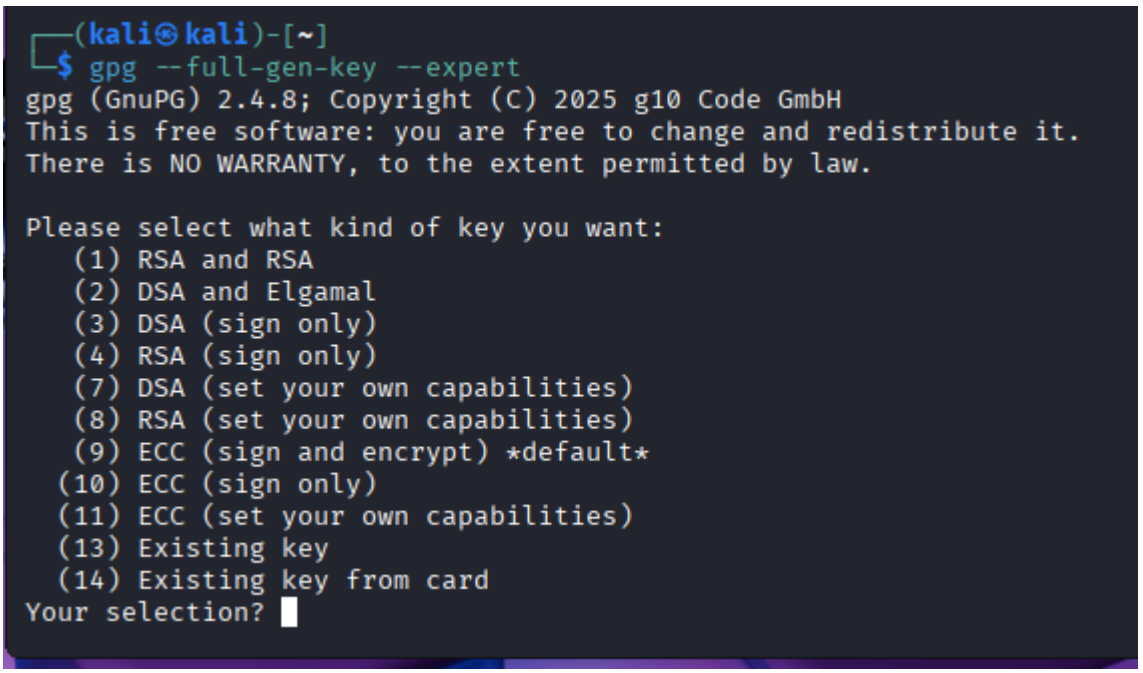
Si decides usar el modo experto para generar la clave de Carlos, podrás elegir entre varios algoritmos de cifrado. Algunas de las opciones incluyen:

- **RSA (firma y cifrado):** es el algoritmo más común y ampliamente soportado.
- **ECC (criptografía de curva elíptica):** ofrece mayor seguridad con claves más pequeñas.
- **DSA/ElGamal:** DSA se usa para firmar digitalmente, y ElGamal para cifrar.

Para generar una clave en modo experto con ECC, por ejemplo, usa el comando:

```
gpg --full-gen-key --expert
```

Aquí puedes elegir la opción de ECC para usar curvas elípticas.



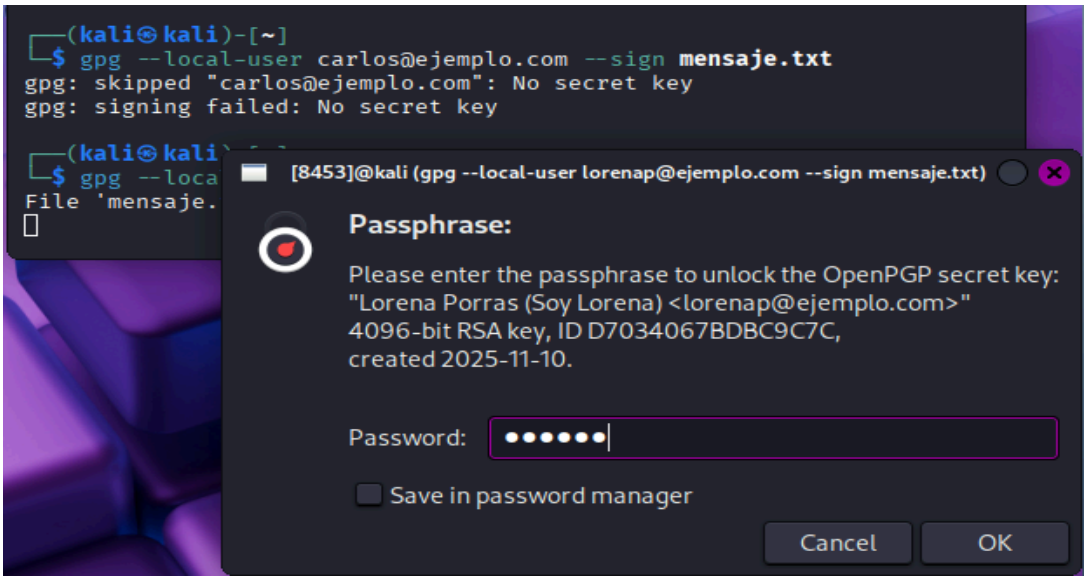

---

### 8. Firmar un archivo o mensaje

Para garantizar la autenticidad y no repudio del remitente, puedes firmar un archivo con tu clave privada. Supongamos que deseas firmar un archivo llamado mensaje.txt, puedes hacerlo ejecutando:

```
gpg --local-user carlos@ejemplo.com --sign mensaje.txt
```

Esto creará un archivo mensaje.txt.gpg que contiene tanto el archivo original como la firma digital.



```
┌──(kali㉿kali)-[~]
└─$ gpg --local-user carlos@ejemplo.com --sign mensaje.txt
gpg: skipped "carlos@ejemplo.com": No secret key
gpg: signing failed: No secret key

┌──(kali㉿kali)-[~]
└─$ gpg --local-user lorenap@ejemplo.com --sign mensaje.txt
File 'mensaje.txt.gpg' exists. Overwrite? (y/N) y
```

---

### 9. Verificación de la firma

Para verificar la autenticidad del archivo y asegurarte de que no ha sido alterado, el destinatario puede usar el siguiente comando:

```
gpg --verify mensaje.txt.gpg
```

Este comando permitirá comprobar que el archivo proviene del remitente esperado y que no ha sido modificado en el proceso.

```
┌──(kali㉿kali)-[~]
└─$ gpg --verify mensaje.txt.gpg
gpg: Signature made Mon 10 Nov 2025 06:31:36 AM EST
gpg:                using RSA key D9D9E212170D61B2016AFC2BD7034067BD
BC9C7C
gpg:                issuer "lorenap@ejemplo.com"
gpg: Good signature from "Lorena Porras (Soy Lorena) <lorenap@ejempl
o.com>" [ultimate]

┌──(kali㉿kali)-[~]
└─$
```

---

**Casos de uso**

1. **Protección de correos electrónicos:**
   Uno de los usos más comunes de PGP y GPG es en la encriptación y firma de correos electrónicos.
   Herramientas como **Enigmail** (para Thunderbird) o **Mailvelope** (para webmail) permiten esta integración.
2. **Firma de documentos:**
   Se utiliza ampliamente para la firma digital de documentos.
   Para transacciones financieras y otros propósitos que permiten confirmar acuerdos o autorizaciones sin necesidad de firmas manuscritas.
3. **Protección de archivos y datos personales:**
   Los usuarios pueden cifrar archivos sensibles, como registros médicos, documentos fiscales o cualquier otro dato privado, antes de almacenarlos o compartirlos.

4. **Verificación de software:**
   Permite a los usuarios verificar la autenticidad del software que están instalando, asegurándose de que no ha sido manipulado por terceros malintencionados.
   Los proyectos de código abierto suelen firmar sus versiones de software para que los desarrolladores y usuarios puedan verificar la autenticidad del código.
5. **Redes privadas y comunicaciones seguras:**
   Es utilizado en la creación de túneles **VPN (Virtual Private Network)** y otros tipos de comunicaciones seguras.

---

---

# Ejemplos prácticos con CryptoLoop

CryptoLoop es una herramienta en Linux que permite cifrar sistemas de archivos, utilizando un cifrado por bloques para proteger la integridad y confidencialidad de los datos almacenados. A diferencia de PGP y GPG, que se centran en la encriptación de archivos individuales o comunicaciones, CryptoLoop se usa principalmente para cifrar volúmenes completos, como discos duros o particiones.

Se basa en el concepto de cifrado de bloques, donde los datos se dividen en bloques de un tamaño específico antes de ser cifrados y almacenados en el disco. Esto garantiza que cada bloque de datos sea cifrado independientemente, haciendo más difícil para un atacante reconstruir el contenido original del archivo o sistema de archivos completo.

---

**Uso de CryptoLoop en Linux**

**1. Preparativos**

Antes de comenzar con la implementación de CryptoLoop, es importante tener algunos conocimientos básicos sobre particiones y discos en Linux, así como privilegios de superusuario (**root**) para realizar las configuraciones necesarias.
Se recomienda realizar la práctica en la nube o en físico sobre una máquina virtual de Linux para evitar dañar o perder datos sensibles.

---

**1.1. Verificación de módulos**

Verifica si los módulos necesarios están cargados en el núcleo del sistema operativo:

```
lsmod | grep loop
```

```
┌──(kali㉿kali)-[~]
└─$ lsmod | grep loop
vsock_loopback         12288  0
vmw_vsock_virtio_transport_common    61440  1 vsock_loopback
vsock                  73728  5 vmw_vsock_virtio_transport_common,vs
ock_loopback,vmw_vsock_vmci_transport
```

Si no están cargados, puedes cargarlos manualmente:

```
modprobe loop

modprobe cryptoloop
```

```
┌──(kali㉿kali)-[~]
└─$ sudo modprobe loop
[sudo] password for kali:

┌──(kali㉿kali)-[~]
└─$ sudo modprobe cryptoloop
modprobe: FATAL: Module cryptoloop not found in directory /lib/modul
es/6.16.8+kali-amd64

┌──(kali㉿kali)-[~]
└─$
```

### 1.2. Instalación de utilidades

Asegúrate de tener instaladas las utilidades **losetup** y **cryptsetup**, que son necesarias para configurar dispositivos de bucle (*loop devices*) y manejar el cifrado.
En la mayoría de las distribuciones de Linux, estas herramientas están disponibles en los repositorios oficiales y se pueden instalar con:

```
sudo apt-get install cryptsetup
```

```
┌──(kali㉿kali)-[~]
└─$ sudo apt-get install cryptsetup
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
cryptsetup is already the newest version (2:2.8.1-1).
The following package was automatically installed and is no longer r
equired:
  python3-proton-vpn-lib
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 72 not upgraded.
```

### 2. Ejemplo práctico de configuración con CryptoLoop

A continuación, cifraremos una partición o archivo contenedor utilizando CryptoLoop.

---

### 2.1. Crear un archivo contenedor

Primero, creamos un archivo que actuará como un contenedor cifrado:

```
dd if=/dev/zero of=/home/usuario/mi_volumen_cifrado bs=1M count=100
```

Este comando crea un archivo de 100 MB lleno de ceros.

```
┌──(kali㉿kali)-[~]
└─$ sudo dd if=/dev/zero of=/home/usuario/mi_volumen_cifrado bs=1M c
ount=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.0921739 s, 1.1 GB/s
```

### 2.2. Asociar el archivo a un dispositivo de bucle

El archivo creado se asocia con un dispositivo de bloques para que el sistema operativo pueda tratarlo como un dispositivo de bloque real:

```
losetup /dev/loop0 /home/usuario/mi_volumen_cifrado
```

```
┌──(kali㉿kali)-[~]
└─$ sudo losetup /dev/loop0 /home/usuario/mi_volumen_cifrado
```

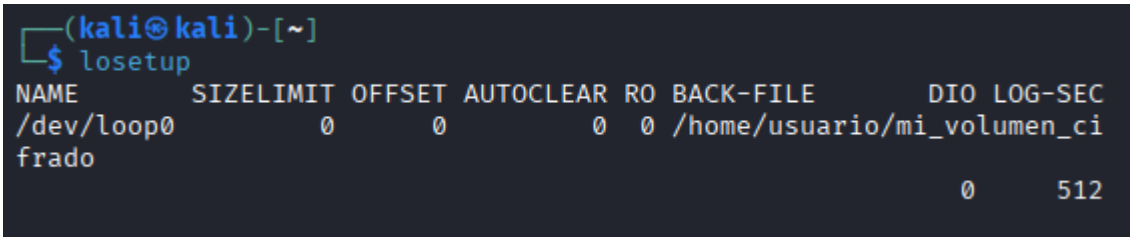
Si aparece el error:

```
losetup: /home/parallels/borrar: failed to set up loop device: Device or
resource busy
```

utilizar el comando:

```
losetup
```

Nos mostrará una lista similar a la de la imagen.
Buscar un **loop** que no esté en uso y usar ese número, por ejemplo:
loop12.




### 2.3. Configurar el cifrado con CryptoLoop

Ahora se configura el cifrado sobre el dispositivo de bucle usando CryptoLoop.
Aquí se usa **AES** como algoritmo de cifrado:

cryptsetup -c aes -h sha256 y create volumen_cifrado /dev/loop0

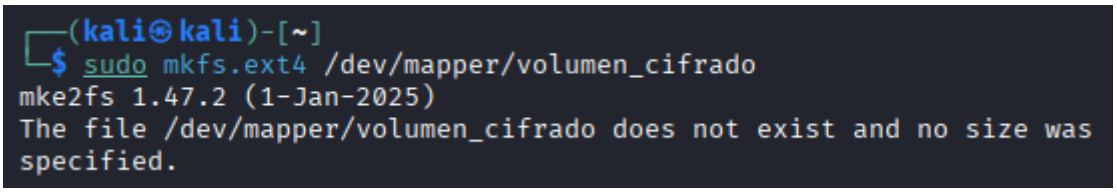
- -c aes: especifica el algoritmo de cifrado.
- -h sha256: indica el algoritmo hash para la clave.
- -y: pide la confirmación de la clave antes de proceder.

---

### 2.4. Formatear el volumen cifrado

Una vez configurado el cifrado, se debe formatear el volumen para usarlo como un sistema de archivos:

```
mkfs.ext4 /dev/mapper/volumen_cifrado
```




---

### 2.5. Montar el volumen cifrado

Finalmente, se monta el volumen para que pueda ser utilizado como cualquier otro sistema de archivos:

```
mount /dev/mapper/volumen_cifrado /mnt
```

Ahora, el sistema está listo para almacenar datos cifrados en /mnt.

```
┌──(kali㉿kali)-[~]
└─$ sudo mount /dev/mapper/volumen_cifrado /mnt
mount: /mnt: fsconfig() failed: /dev/mapper/volumen_cifrado: Can't l
ookup blockdev.
       dmesg(1) may have more information after failed mount system
call.
```

---

### 2.6. Desmontar y cerrar el volumen

Para desmontar y cerrar el volumen cifrado cuando se haya terminado de usar:

```
umount /mnt

cryptsetup remove volumen_cifrado

losetup -d /dev/loop0
```

```
┌──(kali㉿kali)-[~]
└─$ sudo umount /mnt
umount: /mnt: not mounted.

┌──(kali㉿kali)-[~]
└─$ umount /mnt
umount: /mnt: not mounted.

┌──(kali㉿kali)-[~]
└─$ cryptsetup remove volumen_cifrado
Cannot initialize device-mapper, running as non-root user.

┌──(kali㉿kali)-[~]
└─$ sudo cryptsetup remove volumen_cifrado
Device volumen_cifrado is not active.

┌──(kali㉿kali)-[~]
└─$ losetup -d /dev/loop0
losetup: /dev/loop0: detach failed: Permission denied

┌──(kali㉿kali)-[~]
└─$ sudo losetup -d /dev/loop0

┌──(kali㉿kali)-[~]
└─$
```

---

### 3. Volver a montar el volumen cifrado

Después de haber desmontado y cerrado el volumen cifrado, si deseas volver a montarlo en el futuro, debes repetir algunos de los pasos previos.
Para ello, debes asociar el archivo de volumen cifrado con un dispositivo de bucle y reconfigurar el cifrado con la misma clave que usaste anteriormente. Los pasos son los siguientes:

---

#### 3.1. Asociar el archivo cifrado con un dispositivo de bucle

Primero, debes volver a asociar el archivo contenedor cifrado con un dispositivo de bucle:

**losetup /dev/loop0 /home/parallels/mi_volumen_cifrado**

Este comando asocia el archivo mi_volumen_cifrado con el dispositivo de bucle /dev/loop0.

---

#### 3.2. Configurar el cifrado con la misma contraseña

Luego, debes volver a configurar el cifrado usando el mismo algoritmo y contraseña que usaste cuando lo creaste. Se te pedirá que ingreses la contraseña original:

```
cryptsetup -c aes -h sha256 create volumen_cifrado /dev/loop0
```

---

```
┌──(kali㉿kali)-[~]
└─$ cryptsetup -c aes -h sha256 create volumen_cifrado /dev/loop0
WARNING: Using default options for cipher (aes-cbc-plain, key size 2
56 bits) that could be incompatible with older versions.
For plain mode, always use options --cipher, --key-size and if no ke
yfile or keyring is used, then also --hash.
Device /dev/loop0 does not exist or access denied.
```

#### 3.3. Montar el volumen cifrado

Después de introducir la contraseña correcta, puedes montar el volumen cifrado como lo hiciste anteriormente:

```
mount /dev/mapper/volumen_cifrado /mnt
```

```
┌──(kali㉿kali)-[~]
└─$ mount /dev/mapper/volumen_cifrado /mnt
mount: /mnt: must be superuser to use mount.
       dmesg(1) may have more information after failed mount system
call.
```

Ahora, si ejecutas:

```
ls /mnt
```

Podrás ver y acceder a los archivos almacenados en el volumen cifrado.
Recuerda que estos pasos deben repetirse cada vez que desees volver a montar el volumen cifrado después de haberlo desmontado y cerrado.
Asegúrate de usar la misma contraseña que configuraste en el paso original.

```
┌──(kali㉿kali)-[~]
└─$ ls /mnt
labimg
```

---

**Conclusión**

CryptoLoop se puede usar para proteger archivos y datos sensibles en discos o discos externos. Cifrando un volumen completo, los datos se mantienen protegidos incluso si el dispositivo es robado o perdido.
En ambientes de desarrollo o pruebas, donde los datos sensibles no deben estar accesibles fácilmente, se puede usar para crear volúmenes cifrados que solo los desarrolladores autorizados puedan montar y acceder. También para realizar copias de seguridad que contengan información confidencial, permitiendo cifrar esas copias antes de almacenarlas en ubicaciones externas o en la nube, asegurando que los datos estén protegidos en caso de violaciones de seguridad.

Aunque es una herramienta útil para entender los fundamentos del cifrado de volúmenes en Linux, es importante señalar que ha sido reemplazada en gran medida por tecnologías más modernas como **LUKS** y **dm-crypt**, que ofrecen mayor seguridad, funcionalidades avanzadas y soporte activo.
Sin embargo, la comprensión de CryptoLoop proporciona una base sólida para aquellos que deseen profundizar en la criptografía aplicada al cifrado de discos y volúmenes en sistemas operativos basados en Linux.

**LUKS** se ha convertido en el estándar para el cifrado de discos en sistemas Linux debido a sus ventajas frente a soluciones anteriores.

---

**ChatGPT Plus**

Perfecto 👍 Aquí tienes transcrito en texto todo el contenido de las dos páginas sobre **LUKS en Linux**:

---

**Uso de LUKS en Linux**

**1. Introducción**

Usaremos **LUKS** para cifrar y gestionar volúmenes en un entorno Linux, proporcionando una solución más avanzada y segura que **CryptoLoop**.
Además, aprenderemos a configurar, montar y desmontar volúmenes cifrados, gestionando las claves y metadatos asociados, y entendiendo el flujo completo de creación y acceso a un volumen cifrado con LUKS.

---

**2. Preparativos**

Antes de comenzar con la implementación de LUKS, asegúrate de tener privilegios de superusuario (**root**) para realizar las configuraciones necesarias.

---

**3. Instalación de las utilidades**

Asegúrate de tener instaladas las utilidades **cryptsetup**, que son necesarias para manejar el cifrado.
En la mayoría de las distribuciones de Linux, estas herramientas están disponibles en los repositorios oficiales y se pueden instalar con:

```
sudo apt-get install cryptsetup
```

---

**4. Configuración**

**4.1. Crear un archivo contenedor**

Primero, creamos un archivo que actuará como un contenedor cifrado:

```
dd if=/dev/zero of=/home/usuario/mi_volumen_cifrado bs=1M count=100
```

Este comando crea un archivo de **100 MB** lleno de ceros.

---

**4.2. Configurar el cifrado con LUKS**

Ahora configuramos el cifrado sobre el archivo contenedor usando LUKS:

```
sudo cryptsetup luksFormat /home/usuario/mi_volumen_cifrado
```

Se le pedirá que confirme y que introduzca una contraseña para el cifrado.

---

**4.3. Abrir el volumen cifrado**

Después de configurar el cifrado, debemos abrir el volumen para acceder a él:

```
sudo cryptsetup luksOpen /home/usuario/mi_volumen_cifrado volumen_cifrado
```

---

**4.4. Formatear el volumen cifrado**

Una vez abierto el volumen, se debe formatear para usarlo como un sistema de archivos:

```
sudo mkfs.ext4 /dev/mapper/volumen_cifrado
```

---

**4.5. Montar el volumen cifrado**

Finalmente, montamos el volumen para que pueda ser utilizado como cualquier otro sistema de archivos:

```
sudo mount /dev/mapper/volumen_cifrado /mnt
```

Ahora, el sistema está listo para almacenar datos cifrados en /mnt.

---

**5. Desmontar y cerrar el volumen**

Para desmontar y cerrar el volumen cifrado cuando haya terminado de usarse:

```
sudo umount /mnt

sudo cryptsetup luksClose volumen_cifrado
```

---

**6. Volver a montar el volumen**

Si deseas volver a cargar el volumen cifrado después de haberlo desmontado, debes repetir los pasos de apertura y montaje, introduciendo la contraseña anterior:

```
sudo cryptsetup luksOpen /home/usuario/mi_volumen_cifrado volumen_cifrado
```

```
sudo mount /dev/mapper/volumen_cifrado /mnt
```

**LUKS nos ofrece:**

- **Protección de datos sensibles:** se utiliza para proteger archivos y datos sensibles en dispositivos portátiles o discos externos.
- **Entornos de desarrollo y pruebas:** ideal para crear volúmenes cifrados donde se manipulen datos confidenciales o de proyectos.
- **Gestión avanzada de claves:** permite añadir múltiples contraseñas o claves de recuperación, lo que facilita la administración segura del volumen.
- **Compatibilidad con dm-crypt:** LUKS se integra con el subsistema de cifrado del kernel de Linux, garantizando un alto rendimiento y estabilidad.

# En el caso de que haya error

